

Cours Theorique

Seance 13 : Testing en React

Module M204 - Developpement Front-End

Objectifs :

- Comprendre l'importance des tests
- Maitriser Jest pour les tests unitaires
- Utiliser React Testing Library
- Tester composants, events et state

Table des matières

1	Introduction aux Tests	2
1.1	Pourquoi Tester ?	2
1.2	Types de Tests	2
1.3	Outils de Test React	2
2	Jest - Framework de Test	3
2.1	Installation	3
2.2	Structure d'un Test	3
2.3	Assertions Courantes	4
2.4	Tester une Fonction	4
3	React Testing Library	5
3.1	Philosophie	5
3.2	Methodes de Requete	5
3.3	Premier Test de Composant	5
3.4	Tester les Evenements	6
3.5	Tester un Composant avec State	6
3.6	Tester un Formulaire	8
3.7	Tester avec des Props	10
4	Recapitulatif	11
4.1	Structure des Fichiers de Test	11
4.2	Commandes	11
4.3	Bonnes Pratiques	11
4.4	Checklist des Matchers	11

1 Introduction aux Tests

1.1 Pourquoi Tester ?

- **Fiabilité** : Les tests détectent les bugs tôt et évitent les régressions
- **Documentation** : Les tests décrivent le comportement attendu
- **Collaboration** : Facilite le travail en équipe

1.2 Types de Tests

Type	Description
Unitaire	Teste une fonction/composant isolé
Integration	Teste l'interaction entre composants
End-to-End (E2E)	Teste l'application complète (Cypress, Playwright)

1.3 Outils de Test React

- **Jest** : Framework de test (assertions, mocks, coverage)
- **React Testing Library** : Teste les composants comme un utilisateur
- **Vitest** : Alternative moderne à Jest (plus rapide)

Important

Enzyme (Airbnb) n'est plus maintenu et incompatible avec React 18. Utilisez **React Testing Library** pour les nouveaux projets.

2 Jest - Framework de Test

2.1 Installation

Avec Vite, installer Vitest (compatible Jest) :

```
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom
```

vite.config.js :

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    globals: true,
    environment: 'jsdom',
    setupFiles: './src/setupTests.js',
  },
})
```

src/setupTests.js :

```
import '@testing-library/jest-dom'
```

package.json :

```
"scripts": {
  "test": "vitest",
  "test:run": "vitest run"
}
```

2.2 Structure d'un Test

```
import { describe, it, expect } from 'vitest'

describe('Nom du groupe de tests', () => {
  it('devrait faire quelque chose', () => {
    // Arrange (preparation)
    const a = 2
    const b = 3

    // Act (action)
    const result = a + b

    // Assert (verification)
    expect(result).toBe(5)
  })
})
```

2.3 Assertions Courantes

Matcher	Description
toBe(value)	Egalite stricte (===)
toEqual(value)	Egalite profonde (objets/arrays)
toBeTruthy()	Valeur truthy
toBeFalsy()	Valeur falsy
toContain(item)	Array contient item
toHaveLength(n)	Longueur = n
toThrow()	Lance une erreur
toBeNull()	Est null

```
// Exemples
expect(5).toBe(5)
expect({ name: 'Ali' }).toEqual({ name: 'Ali' })
expect([1, 2, 3]).toContain(2)
expect([1, 2, 3]).toHaveLength(3)
expect(true).toBeTruthy()
expect(null).toBeNull()
```

2.4 Tester une Fonction

```
// utils.js
export function formatPrice(price) {
  return `${price} DH`
}

export function calculateTotal(items) {
  return items.reduce((sum, item) => sum + item.price, 0)
}
```

```
// utils.test.js
import { describe, it, expect } from 'vitest'
import { formatPrice, calculateTotal } from './utils'

describe('formatPrice', () => {
  it('devrait formater le prix avec DH', () => {
    expect(formatPrice(100)).toBe('100 DH')
  })
})

describe('calculateTotal', () => {
  it('devrait calculer le total des items', () => {
    const items = [{ price: 100 }, { price: 200 }]
    expect(calculateTotal(items)).toBe(300)
  })

  it('devrait retourner 0 pour un array vide', () => {
    expect(calculateTotal([])).toBe(0)
  })
})
```

3 React Testing Library

3.1 Philosophie

Testing

"Plus vos tests ressemblent a la facon dont votre logiciel est utilise, plus ils vous donnent confiance." - Kent C. Dodds

React Testing Library teste les composants **comme un utilisateur** :

- Chercher par texte visible, role, label
- Eviter de tester les details d'implementation
- Simuler les interactions utilisateur

3.2 Methodes de Requete

Methode	Usage
getByText	Trouve par texte visible
getByRole	Trouve par role (button, heading...)
getByLabelText	Trouve input par son label
getByPlaceholderText	Trouve par placeholder
getByTestId	Trouve par data-testid
queryBy...	Retourne null si non trouve
findBy...	Async, attend l'element

3.3 Premier Test de Composant

```
// Button.jsx
function Button({ children, onClick }) {
  return (
    <button onClick={onClick} className="btn">
      {children}
    </button>
  )
}
export default Button
```

```
// Button.test.jsx
import { render, screen } from '@testing-library/react'
import { describe, it, expect } from 'vitest'
import Button from './Button'

describe('Button', () => {
  it('devrait afficher le texte du bouton', () => {
    render(<Button>Cliquez-moi</Button>)

    expect(screen.getByText('Cliquez-moi')).toBeInTheDocument()
  })

  it('devrait avoir le role button', () => {
    render(<Button>Test</Button>)

    expect(screen.getByRole('button')).toBeInTheDocument()
  })
})
```

3.4 Tester les Evenements

```
import { render, screen, fireEvent } from '@testing-library/react'
import { describe, it, expect, vi } from 'vitest'
import Button from './Button'

describe('Button events', () => {
  it('devrait appeler onClick quand clique', () => {
    const handleClick = vi.fn() // Mock function

    render(<Button onClick={handleClick}>Cliquer</Button>)

    fireEvent.click(screen.getByText('Cliquer'))

    expect(handleClick).toHaveBeenCalledTimes(1)
  })
})
```

Astuce

`vi.fn()` cree une fonction mock qui enregistre les appels. Utilisez `toHaveBeenCalledTimes()` ou `toHaveBeenCalledWith(arg)` pour verifier.

3.5 Tester un Composant avec State

```
// Counter.jsx
import { useState } from 'react'

function Counter() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <p>Compteur: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(count - 1)}>-</button>
    </div>
  )
}
export default Counter
```

```
// Counter.test.jsx
import { render, screen, fireEvent } from '@testing-library/react'
import { describe, it, expect } from 'vitest'
import Counter from './Counter'

describe('Counter', () => {
  it('devrait afficher 0 au debut', () => {
    render(<Counter />)
    expect(screen.getByText('Compteur: 0')).toBeInTheDocument()
  })

  it('devrait incrementer quand + clique', () => {
    render(<Counter />)
    fireEvent.click(screen.getByText('+'))
    expect(screen.getByText('Compteur: 1')).toBeInTheDocument()
  })
})
```

```
  })  
  
  it('devrait decrementer quand - clique', () => {  
    render(<Counter />)  
    fireEvent.click(screen.getByText('-'))  
    expect(screen.getByText('Compteur: -1')).toBeInTheDocument()  
  })  
})
```

3.6 Tester un Formulaire

```
// LoginForm.jsx
import { useState } from 'react'

function LoginForm({ onSubmit }) {
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')

  const handleSubmit = (e) => {
    e.preventDefault()
    onSubmit({ email, password })
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        placeholder="Email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      <input
        type="password"
        placeholder="Mot de passe"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <button type="submit">Connexion</button>
    </form>
  )
}
export default LoginForm
```

```
// LoginForm.test.jsx
import { render, screen, fireEvent } from '@testing-library/react'
import { describe, it, expect, vi } from 'vitest'
import LoginForm from './LoginForm'

describe('LoginForm', () => {
  it('devrait soumettre les donnees du formulaire', () => {
    const handleSubmit = vi.fn()
    render(<LoginForm onSubmit={handleSubmit} />)

    // Remplir les champs
    fireEvent.change(screen.getByPlaceholderText('Email'), {
      target: { value: 'test@email.com' }
    })
    fireEvent.change(screen.getByPlaceholderText('Mot de passe'), {
      target: { value: '123456' }
    })

    // Soumettre
    fireEvent.click(screen.getByText('Connexion'))

    // Verifier
    expect(handleSubmit).toHaveBeenCalledWith({
      email: 'test@email.com',
    })
  })
})
```

```
    password: '123456'  
  }  
}  
}
```

3.7 Tester avec des Props

```
// ProductCard.test.jsx
import { render, screen, fireEvent } from '@testing-library/react'
import { describe, it, expect, vi } from 'vitest'
import ProductCard from './ProductCard'

const mockProduct = {
  id: 1,
  name: 'iPhone 15',
  price: 12000,
  category: 'Electronique'
}

describe('ProductCard', () => {
  it('devrait afficher les infos du produit', () => {
    render(<ProductCard product={mockProduct} onDelete={() => {}} />)

    expect(screen.getByText('iPhone 15')).toBeInTheDocument()
    expect(screen.getByText('12000 DH')).toBeInTheDocument()
    expect(screen.getByText('Electronique')).toBeInTheDocument()
  })

  it('devrait appeler onDelete avec id', () => {
    const handleDelete = vi.fn()
    render(<ProductCard product={mockProduct} onDelete={handleDelete} />)

    fireEvent.click(screen.getByText('Supprimer'))

    expect(handleDelete).toHaveBeenCalledOnce()
  })
})
```

4 Recapitulatif

4.1 Structure des Fichiers de Test

```
src/  
  components/  
    Button.jsx  
    Button.test.jsx      <- Test a cote du composant  
    Counter.jsx  
    Counter.test.jsx  
  utils/  
    helpers.js  
    helpers.test.js  
  setupTests.js
```

4.2 Commandes

Commande	Description
npm test	Lance les tests en mode watch
npm run test:run	Lance les tests une fois
npm run test -- --coverage	Affiche la couverture

4.3 Bonnes Pratiques

- Nommer les fichiers `*.test.jsx` ou `*.spec.jsx`
- Tester le comportement, pas l'implémentation
- Un test = une assertion principale
- Utiliser des mocks pour les fonctions externes
- Viser 70-80% de couverture de code

4.4 Checklist des Matchers

Jest/Vitest	Testing Library
toBe, toEqual	toBeInTheDocument
toBeTruthy, toBeFalsy	toBeVisible
toContain	toHaveTextContent
toHaveBeenCalled	toHaveValue
toHaveBeenCalledWith	toBeDisabled